ARTIFICIAL INTELLIGENCE

Comprehensive Lecture Notes

ADP Program | Riphah International College

Covers: Week 1 – 8 (Lectures 1 – 16)

Course: Artificial Intelligence	**Prepared For:** ADP Students
Institution: Riphah International College	

WEEK 1 — Introduction to Artificial Intelligence

Lecture 1 & 2 – What is AI? Goals & Four Basics

1.1 Introduction to Artificial Intelligence

Definition
Artificial Intelligence (AI): The branch of computer science concerned with creating machines or software that exhibit intelligent behaviour — the ability to perceive their environment, reason, learn, and act to achieve goals.

Historical Milestones:

- 1950 – Alan Turing proposes the **Turing Test** as a measure of machine intelligence.
- 1956 – John McCarthy coins the term **"Artificial Intelligence"** at Dartmouth Conference.
- 1980s – Expert Systems boom; knowledge-based programs applied in medicine & finance.
- 2010s–present – Deep Learning revolution; AI surpasses human performance on many benchmarks.

1.2 Goals of Artificial Intelligence

- **Automate cognitive tasks:** replace or augment human reasoning in complex tasks.
- **Build rational agents:** systems that perceive the environment and act to maximise a performance measure.
- **Understand intelligence:** model human thought processes computationally.
- **Solve complex problems:** NLP, computer vision, autonomous driving, medical diagnosis.

1.3 The Four Basics of AI (Russell & Norvig Framework)

AI can be defined along two dimensions: (1) human vs. rational, and (2) thinking vs. acting.

Approach	Description	Example
Acting Humanly	Turing Test approach — a machine acts indistinguishably from a human in conversation.	Chatbots like ChatGPT that mimic human responses.
Thinking Humanly	Cognitive modelling — simulate the internal mental processes of humans.	Expert systems that trace human problem-solving steps.
Thinking Rationally	Laws of thought — logic-based approach; correct inference from facts.	Theorem provers; Prolog programs.

Acting Rationally	Rational agent — act to achieve the best expected outcome given the environment.	A chess engine that selects the best move.

Example

Acting Humanly: A customer service chatbot answers questions so naturally that callers cannot tell it is not human.
Thinking Humanly: A medical expert system follows the same diagnostic steps a doctor would follow.
Thinking Rationally: A program proves mathematical theorems using formal rules of logic.
Acting Rationally: A self-driving car selects the path that minimises travel time and maximises safety.

Practice Questions

1. **Q1:** Define Artificial Intelligence. What distinguishes AI from conventional programming?
2. **Q2:** Explain the Turing Test. What are its limitations as a measure of intelligence?
3. **Q3:** Compare 'Thinking Humanly' with 'Thinking Rationally'. Give one example of each.
4. **Q4:** Which of the four AI approaches is considered most sound by modern AI researchers? Justify your answer.
5. **Q5:** List three real-world applications of AI mentioned in the course outline and classify each under one of the four basics.

WEEK 2 — Foundations of AI & Introduction to Agents

Lecture 3 – Foundations of AI

2.1 Multi-Disciplinary Foundations

AI draws from six major disciplines:

- **Philosophy:** Logic, reasoning, mind-body problem, knowledge representation.
- **Mathematics:** Formal logic, probability, statistics, computation theory, algorithms.
- **Psychology:** Cognitive models of human memory, perception, problem-solving.
- **Linguistics:** Grammar, language understanding, knowledge representation.
- **Computer Engineering:** Fast processors, memory systems enabling large-scale AI.
- **Neuroscience:** Biological neural networks inspire artificial neural networks.

2.2 Programming Without AI vs. With AI

Traditional Programming	AI Programming
Fixed rules written explicitly by the programmer	Rules/behaviour learned from data or inferred by the system
Cannot adapt to unseen input	Can generalise to new situations
Output is deterministic from fixed logic	Output may be probabilistic or context-sensitive
Example: calculator, payroll system	Example: spam filter, speech recogniser

2.3 Key AI Application Areas

- **Gaming:** Chess (Deep Blue), Go (AlphaGo) — AI plays at super-human level.
- **Natural Language Processing (NLP):** Machine translation, sentiment analysis, chatbots.
- **Expert Systems:** MYCIN (medical diagnosis), DENDRAL (chemistry) — encapsulate domain expertise.
- **Computer Vision:** Object detection, facial recognition, medical imaging.
- **Intelligent Robots:** Industrial arms, autonomous vehicles, surgical robots.

Lecture 4 – Artificial Agents

2.4 What is an Agent?

Definition

Agent: Anything that perceives its environment through sensors and acts upon that environment through actuators to achieve goals.

Definition

Percept: The current sensory input to an agent at any given moment.

Definition

Percept Sequence: The complete history of everything the agent has perceived up to the current moment.

2.5 Structure of an Agent

- **Sensors:** Receive input from the environment (camera, microphone, keyboard, temperature sensor).
- **Actuators:** Act on the environment (motors, screen output, speakers, robotic arms).
- **Agent Function:** Maps any percept sequence to an action: f: P* → A
- **Agent Program:** The concrete implementation of the agent function in software.

2.6 Rationality and Omniscience

Definition

Rational Agent: An agent that selects actions that maximise its expected performance measure, given the evidence from its percept sequence and any built-in knowledge.

Definition

Performance Measure: An objective criterion for evaluating the success of an agent's behaviour (e.g., score in a game, distance driven without accident).

Definition

Omniscience: A hypothetical agent that knows the actual outcomes of all its actions. Rationality is NOT the same as omniscience — a rational agent makes the best decision given available information.

2.7 Types of Agent Programs

- **Simple Reflex Agent:** Acts only on current percept using condition-action rules.
- **Model-Based Reflex Agent:** Maintains internal state to handle partially observable environments.
- **Goal-Based Agent:** Uses goal information to choose actions that achieve desired states.
- **Utility-Based Agent:** Uses a utility function to choose the best action among alternatives.

- **Learning Agent:** Improves performance over time through experience.

Example

Simple Reflex: A thermostat — IF temperature < 18°C THEN turn on heater.
Model-Based: A robot vacuum that builds a map of the room it has cleaned.
Goal-Based: A navigation system that plans routes to reach a destination.
Utility-Based: An airline booking agent that maximises comfort AND minimises cost.

Practice Questions

6. **Q1:** Define an agent and differentiate between a sensor and an actuator.
7. **Q2:** What is the difference between an agent function and an agent program?
8. **Q3:** Explain rationality. Why is a rational agent NOT necessarily omniscient?
9. **Q4:** List and briefly describe the five types of agent programs.
10. **Q5:** Give a real-world example of a utility-based agent and describe its performance measure.

WEEK 3 — Agent Architectures & AI Environments

Lecture 5 – Agent Architectures

3.1 Simple Reflex Agent

Definition

Simple Reflex Agent: Selects actions based solely on the current percept, ignoring the rest of the percept history. Uses a set of condition-action (if-then) rules.

Architecture: Percept → Condition Matching → Action

- Easy to implement; fast decision making.
- Works well only in fully observable environments.
- **Limitation:** Cannot handle partial observability; loops in unseen states.

Example

A car's automatic braking system: IF obstacle_detected THEN apply_brakes.

3.2 Model-Based Reflex Agent

Definition

Model-Based Reflex Agent: Maintains an internal state (model of the world) to track aspects of the environment that cannot be perceived directly. The internal state is updated using knowledge of how the world evolves and what effect the agent's own actions have.

- Handles partially observable environments.
- Requires: (1) a model of how the world changes, (2) knowledge of the effect of actions.

Example

A robot that builds a map while exploring, then uses the map to navigate even when sensors cannot see every room.

3.3 Goal-Based Agent

Definition

Goal-Based Agent: Has goal information describing desirable states and chooses actions that will lead to achieving those goals. Combines current state knowledge with search/planning algorithms.

- More flexible than reflex agents — behaviour can change when the goal changes.
- Requires search and planning mechanisms.

Example

A GPS navigation system whose goal is 'reach destination'. It plans the route and re-plans if blocked.

3.4 Utility-Based Agent

Definition

Utility-Based Agent: Uses a utility function that maps a state (or sequence of states) to a real number expressing the agent's happiness or preference. The agent selects actions that maximise expected utility.

- Handles conflicting goals and trade-offs.
- Provides a precise performance measure when there are multiple possible goals.

Example

A stock-trading AI that trades off risk vs. return by assigning utility to portfolio states.

Practice Questions

11. **Q1:** Draw and explain the architecture of a Simple Reflex Agent.
12. **Q2:** How does a Model-Based Agent differ from a Simple Reflex Agent? When is it necessary?
13. **Q3:** Describe the Goal-Based Agent. What additional capabilities does it require compared to reflex agents?
14. **Q4:** What is a utility function? Why is utility-based reasoning superior to simple goal-based reasoning?
15. **Q5:** An autonomous drone must deliver packages while avoiding buildings and managing battery life. Which agent type is most appropriate? Justify your answer.

Lecture 6 – AI Environments

3.5 What is the Task Environment?

Definition

Task Environment: The problem to which an agent is a solution. Described using the PEAS framework: Performance measure, Environment, Actuators, Sensors.

Example PEAS – Self-Driving Car:

Performance	Safety, speed, legality, comfort, fuel efficiency
Environment	Roads, traffic, pedestrians, weather, signage
Actuators	Steering, accelerator, brakes, horn, indicators
Sensors	Camera, GPS, speedometer, radar, lidar

3.6 Properties of Environments

Definition

Accessible vs. Inaccessible: Accessible (Fully Observable): The agent has complete, accurate access to the full state of the environment at every moment. Inaccessible (Partially Observable): The agent's sensors give incomplete or noisy information.

Example

Accessible: Chess — the agent can see the entire board.
Inaccessible: Poker — the agent cannot see the opponents' cards.

Definition

Deterministic vs. Non-Deterministic: Deterministic: The next state of the environment is completely determined by the current state and the agent's action. Non-Deterministic (Stochastic): The next state is not fully predictable; randomness or uncertainty is involved.

Example

Deterministic: A simple calculator — the same input always gives the same output.
Non-Deterministic: Weather prediction — multiple outcomes are possible from the same conditions.

Definition

Episodic vs. Non-Episodic (Sequential): Episodic: The agent's experience is divided into independent episodes; each action depends only on the current percept. Non-Episodic (Sequential): Past actions can affect future decisions.

Example

Episodic: A defect-detection system on an assembly line — each item is independent.
Non-Episodic: Chess — every move affects all future moves.

Definition

Static vs. Dynamic: Static: The environment does not change while the agent is deliberating. Dynamic: The environment can change while the agent is choosing an action.

Example

Static: Crossword puzzle — the puzzle doesn't change while you think.
Dynamic: A taxi driving — traffic changes continuously while the driver plans.

Practice Questions

16. **Q1:** What does the PEAS framework stand for? Apply it to describe a medical diagnosis AI system.
17. **Q2:** Distinguish between a deterministic and a stochastic environment. Give one example of each.
18. **Q3:** Why is a partially observable environment more challenging for an AI agent?
19. **Q4:** Compare episodic and sequential environments. How does the distinction affect agent design?
20. **Q5:** Classify the following environments as static or dynamic: (a) solving a Sudoku puzzle, (b) playing real-time video game, (c) an online chess game with time limits.

WEEK 4 — Environments (cont.) & Knowledge-Based Agents

Lecture 7 – Discrete, Continuous & Environment Classification

4.1 Discrete vs. Continuous Environment

Definition

Discrete Environment: The environment has a finite number of distinct, clearly defined states, percepts, and actions.

Definition

Continuous Environment: The state, percepts, and/or actions are defined by real-valued variables that can take infinitely many values.

Example

Discrete: Chess — finite number of board positions, discrete moves.
Continuous: Self-driving car — speed, steering angle, position are all continuous values.

4.2 Environment Classification Summary Table

Environment	Chess	Taxi Driving	Image Classification
Accessible	Yes	No	Yes (image given)
Deterministic	Yes	No	Yes
Episodic	No	No	Yes
Static	Yes	No	Yes
Discrete	Yes	No	Yes

Practice Questions

21. **Q1:** Classify the environment of an online chess game across all five properties (accessible, deterministic, episodic, static, discrete).
22. **Q2:** A factory robot detects defects in products. Is its environment discrete or continuous? Episodic or sequential? Justify.
23. **Q3:** Why are continuous environments harder for AI agents to handle than discrete ones?

Lecture 8 – Knowledge-Based Agents (KBA)

4.3 Introduction to Knowledge-Based Agents

Definition

Knowledge-Based Agent (KBA): An agent that maintains an explicit, symbolic representation of the world — its knowledge base — and uses logical inference to decide what actions to take.

4.4 Architecture of a Knowledge-Based Agent

- **Knowledge Base (KB):** A set of sentences in a formal knowledge representation language (facts, rules, assertions).
- **Inference Engine:** Derives new sentences from the KB and answers questions using logical deduction.
- **TELL operation:** Adds new percepts/facts to the KB.
- **ASK operation:** Queries the KB to decide what action to take.

Example

A medical diagnosis system TELLs the KB 'Patient has fever AND cough'. It ASKs 'What disease might this be?' and the KB infers 'Possibly influenza' using stored rules.

4.5 Levels of a Knowledge-Based Agent

Definition

Knowledge Level: The highest level — describes what the agent knows and what goals it has, independent of implementation.

Definition

Logical Level: The level at which knowledge is encoded as logical sentences (propositions, predicates).

Definition

Implementation Level: The physical data structures and algorithms that implement the logical-level representations in software.

Practice Questions

24. **Q1:** Define a Knowledge-Based Agent. How does it differ from a Simple Reflex Agent?
25. **Q2:** Explain the TELL and ASK operations of a KBA.
26. **Q3:** Describe the three levels of a KBA. Give an analogy to illustrate each level.

27. **Q4:** What is the role of the inference engine in a knowledge-based agent?
28. **Q5:** Compare the declarative and procedural approaches to designing a KBA.

WEEK 5 — KBA Design, Logic Basics & Propositional Logic

Lecture 9 – Approaches to KBA Design & Logic Concepts

5.1 Declarative vs. Procedural Approach

Definition

Declarative Approach: Knowledge is expressed as facts and rules in a formal language. The system figures out how to use the knowledge to draw conclusions. Example: Prolog programs, expert system rule bases.

Definition

Procedural Approach: Knowledge is encoded directly in procedures (algorithms) that specify step-by-step how to achieve goals. Example: Hard-coded game AI routines.

Declarative	Procedural
Easy to modify/add knowledge	Hard to modify — code must be rewritten
Knowledge is inspectable/explainable	Logic is hidden inside procedures
Slower due to inference overhead	Faster for specific tasks
Example: Prolog, CLIPS expert systems	Example: hand-coded chess engine heuristics

5.2 Syntax and Semantics

Definition

Syntax: The formal rules governing the structure of sentences in a knowledge representation language — what constitutes a legal, well-formed expression.

Definition

Semantics: The rules that define the meaning (truth value) of sentences — the relationship between sentences and the possible worlds they describe.

Example

Syntax: In propositional logic, 'P AND Q' is well-formed; 'AND P Q' may or may not be depending on the language.

Semantics: 'P AND Q' is TRUE only when both P and Q are individually TRUE.

Lecture 10 – Propositional Logic

5.3 Propositional Logic Basics

Definition

Proposition: A declarative statement that is either TRUE or FALSE, but not both. Represented by symbols such as P, Q, R.

Logical Connectives:

Connective	Symbol	Name	Meaning	Example
Negation	¬	NOT	Flips truth value	¬P
Conjunction	∧	AND	True if both true	P ∧ Q
Disjunction	∨	OR	True if at least one true	P ∨ Q
Implication	→	IF...THEN	False only if P true and Q false	P → Q
Biconditional	↔	IFF	True if both same truth value	P ↔ Q

5.4 Rules of Inference

Definition

Rule of Implication (P → Q): If P is true and P implies Q, then Q must be true.

Definition

Converse: The converse of P → Q is Q → P. Note: the converse is NOT logically equivalent to the original.

Definition

Contrapositive: The contrapositive of P → Q is ¬Q → ¬P. The contrapositive IS logically equivalent to the original.

Definition

Inverse: The inverse of P → Q is ¬P → ¬Q. The inverse is NOT logically equivalent to the original.

Example

P → Q: 'If it rains, then the ground is wet.'
Converse (Q → P): 'If the ground is wet, then it rained.' (NOT necessarily true — sprinklers could have run.)
Contrapositive (¬Q → ¬P): 'If the ground is NOT wet, then it did NOT rain.' (Logically equivalent to original.)
Inverse (¬P → ¬Q): 'If it does NOT rain, then the ground is NOT wet.' (NOT necessarily true.)

Practice Questions

29. **Q1:** What is propositional logic? List all five logical connectives with their symbols and truth conditions.
30. **Q2:** Write the contrapositive, converse, and inverse of: 'If the CPU is faulty, the computer will crash.'
31. **Q3:** Construct the full truth table for: (P → Q) ∧ (¬P ∨ Q).
32. **Q4:** Why is the contrapositive logically equivalent to the original implication but the converse is not?
33. **Q5:** Represent the following in propositional logic: 'If it is raining AND the umbrella is missing, then the person will get wet.'

WEEK 6 — Inference Rules in Propositional Logic

Lectures 11 & 12 – Inference Rules

6.1 Modus Ponens

Definition
Modus Ponens: If P → Q is true, and P is true, then Q is true. The most fundamental inference rule.

Form	Example
P → Q P **∴ Q**	If it rains, the ground gets wet. It is raining. **∴ The ground is wet.**

6.2 Modus Tollens

Definition
Modus Tollens: If P → Q is true, and Q is false (¬Q), then P must also be false (¬P).

Form	Example
P → Q ¬Q **∴ ¬P**	If it rains, ground gets wet. Ground is NOT wet. **∴ It did NOT rain.**

6.3 Hypothetical Syllogism

Definition
Hypothetical Syllogism: If P → Q and Q → R, then P → R. Allows chaining of implications.

Example
P → Q: 'If you study hard, you pass the exam.' Q → R: 'If you pass the exam, you get a degree.' ∴ P → R: 'If you study hard, you get a degree.'

6.4 Disjunctive Syllogism

Definition

Disjunctive Syllogism: If P ∨ Q is true and ¬P (P is false), then Q must be true.

Example

P ∨ Q: 'Either the server is down OR the internet is out.'
¬P: 'The server is NOT down.'
∴ Q: 'The internet is out.'

6.5 Addition Rule

Definition

Addition Rule: If P is true, then P ∨ Q is true (for any Q). You can always add a disjunct.

Example

P: 'It is raining.' ∴ P ∨ Q: 'It is raining OR the sun is shining.' (valid regardless of Q's truth)

6.6 Simplification Rule

Definition

Simplification Rule: If P ∧ Q is true, then P is true (and separately, Q is true).

Example

P ∧ Q: 'The file exists AND is readable.' ∴ P: 'The file exists.' (valid inference)

6.7 Resolution Rule

Definition

Resolution Rule: From (P ∨ Q) and (¬P ∨ R), infer (Q ∨ R). The foundation of automated theorem proving.

Example

(P ∨ Q): 'Either it is day OR the lights are on.'
(¬P ∨ R): 'Either it is NOT day OR the alarm is off.'
∴ (Q ∨ R): 'Either the lights are on OR the alarm is off.'

6.8 Introduction to First-Order Logic (FOL)

Definition

First-Order Logic (FOL): Also called Predicate Logic. Extends propositional logic with variables, predicates, functions, and quantifiers, allowing statements about objects and their relationships.

Why FOL over Propositional Logic?

- Propositional logic: can only express statements about specific, named propositions.
- FOL: can express statements about ALL objects, SOME objects, functions, and relationships.

Syntax of FOL:

- **Constants:** Specific objects — e.g., John, 5, RIC
- **Variables:** Placeholders — x, y, z
- **Predicates:** Properties or relations — e.g., Teacher(x), Knows(x, y)
- **Functions:** Map objects to objects — e.g., FatherOf(x)
- **Quantifiers:** ∀ (Universal), ∃ (Existential)

Practice Questions

34. **Q1:** State Modus Ponens and Modus Tollens. How do they differ?
35. **Q2:** Apply Hypothetical Syllogism to the following: 'If the database is corrupt, the query fails.' 'If the query fails, the report is wrong.' What can you conclude?
36. **Q3:** Use Disjunctive Syllogism: 'Either the power is out OR the generator is faulty.' 'The power is NOT out.' What follows?
37. **Q4:** What is the Resolution Rule? Why is it important in automated reasoning systems?
38. **Q5:** Why is First-Order Logic more expressive than Propositional Logic?

WEEK 7 — First-Order Logic Quantifiers & Problem Solving by Search

Lecture 13 – Quantifiers in FOL

7.1 Atomic and Complex Sentences in FOL

Definition

Atomic Sentence: The most basic sentence in FOL — consists of a predicate applied to a list of terms. Example: Lecturer(Usman), GreaterThan(5, 3).

Definition

Complex Sentence: Formed by combining atomic sentences using logical connectives (¬, ∧, ∨, →, ↔) and quantifiers.

7.2 Universal Quantifier (∀)

Definition

Universal Quantifier (∀): Asserts that a predicate holds for ALL objects in the domain. Read as 'for all'.

Syntax: ∀x P(x) — 'For all x, P(x) is true'

Example

∀x (Bird(x) → HasWings(x)) — 'All birds have wings.'
∀x (Student(x) → MustStudy(x)) — 'Every student must study.'
∀x (Human(x) → Mortal(x)) — 'All humans are mortal.'

7.3 Existential Quantifier (∃)

Definition

Existential Quantifier (∃): Asserts that a predicate holds for AT LEAST ONE object in the domain. Read as 'there exists'.

Syntax: ∃x P(x) — 'There exists some x such that P(x) is true'

Example

∃x (Student(x) ∧ Passed(x)) — 'There exists at least one student who passed.'
∃x (Disease(x) ∧ Incurable(x)) — 'There is at least one incurable disease.'
∃x (Country(x) ∧ HasNoArmy(x)) — 'There exists a country with no army.'

7.4 Key Relationships Between Quantifiers

- **De Morgan for Quantifiers:**
  - ¬∀x P(x) is equivalent to ∃x ¬P(x) [Not all P → Some are not P]
  - ¬∃x P(x) is equivalent to ∀x ¬P(x) [No x has P → All lack P]

Key Point / Exam Tip

Exam Tip: Universal quantifier is usually associated with implication (→); Existential quantifier is usually associated with conjunction (∧).

Practice Questions

39. **Q1:** Translate into FOL: 'All computers in the lab are connected to the internet.'
40. **Q2:** Translate into FOL: 'There exists a student who has passed all exams.'
41. **Q3:** Negate the sentence: ∀x (Cat(x) → HasTail(x)). What does the negation mean in plain English?
42. **Q4:** What is the difference between ∀x (P(x) → Q(x)) and ∀x (P(x) ∧ Q(x))? Give an example showing why the distinction matters.
43. **Q5:** Translate: 'Every AI system that learns from data is a machine learning system.' Using constants: AI_System, LearnFromData, ML_System.

Lecture 14 – Problem Solving Agents & Problem Formulation

7.5 Problem-Solving Agents

Definition

Problem-Solving Agent: A goal-based agent that decides what to do by finding a sequence of actions that leads from the initial state to a goal state. Uses search algorithms.

7.6 Problem Formulation — Five Components

- **Initial State:** The starting configuration of the environment.
- **Actions (Operators):** The set of possible actions the agent can take. ACTIONS(s) returns the set of legal actions in state s.
- **Transition Model:** RESULT(s, a) — describes what state results from taking action a in state s.
- **Goal Test:** Determines whether a given state is a goal state.
- **Path Cost:** A function that assigns a numeric cost to each path (sequence of actions).

Example

Problem: Route from Lahore to Islamabad
Initial State: In Lahore
Actions: Drive to adjacent city
Transition Model: Drive(Lahore, Motorway) → Islamabad
Goal Test: CurrentCity = Islamabad?
Path Cost: Total distance (km) or travel time

7.7 Measuring Performance

- **Completeness:** Is the algorithm guaranteed to find a solution if one exists?
- **Optimality:** Does the algorithm find the least-cost solution?
- **Time Complexity:** How long does the algorithm take (usually measured in nodes expanded)?
- **Space Complexity:** How much memory does the algorithm require?

Practice Questions

44. **Q1:** Formulate the 8-puzzle problem using the five components of problem formulation.
45. **Q2:** What is a solution to a search problem? What distinguishes an optimal solution?
46. **Q3:** Define completeness and optimality of a search algorithm. Are they always achievable together?
47. **Q4:** In a graph search, why must we keep track of visited states?

WEEK 8 — Search Strategies: Uninformed & Informed Search

Lecture 15 – Uninformed Search Strategies

8.1 Uninformed vs. Informed Search

Definition

Uninformed Search (Blind Search): Search strategies that have no additional information about states beyond the problem definition. They do not know if one non-goal state is 'better' than another.

Definition

Informed Search (Heuristic Search): Search strategies that use problem-specific knowledge (heuristics) to guide the search efficiently toward the goal.

8.2 Breadth-First Search (BFS)

Definition

BFS: Expands the shallowest (least-deep) unexpanded node first. Uses a FIFO queue as the frontier.

Property	Value	Notes
Complete	Yes (if branching factor finite)	Always finds a solution if one exists
Optimal	Yes (unit step costs)	Finds shallowest = cheapest if all costs equal
Time	O(b^d)	b=branching factor, d=depth of solution
Space	O(b^d)	Must keep all nodes in memory — major weakness

Example

BFS on a tree with root S, children A,B, and A has children C,D, B has children E,F:
Expansion order: S → A → B → C → D → E → F

8.3 Uniform Cost Search (UCS)

Definition

Uniform Cost Search: Expands the node with the lowest path cost g(n) from the start. Uses a priority queue ordered by path cost. Generalisation of BFS for non-uniform step costs.

- **Complete:** Yes (if step costs ≥ ε > 0)
- **Optimal:** Yes — always finds cheapest solution
- **Time/Space:** O(b^(1+⌊ C*/ε⌋)) — depends on optimal cost C*

Example

Shortest path in a road network where different roads have different distances. UCS will always find the cheapest route.

8.4 Depth-First Search (DFS)

Definition

Depth-First Search: Expands the deepest unexpanded node first. Uses a LIFO stack (or recursion) as the frontier.

- **Complete:** No — can loop forever in infinite state spaces (use graph search to fix)
- **Optimal:** No — does not guarantee the shallowest solution
- **Time:** O(b^m) — m is maximum depth of state space
- **Space:** O(b*m) — only stores the current path and siblings

Example

DFS expansion on same tree: S → A → C (goes deep first before backtracking to D, then B, etc.)

8.5 Iterative Deepening Search (IDS)

Definition

Iterative Deepening DFS: Combines BFS's completeness/optimality with DFS's memory efficiency. Runs DFS with a depth limit; if no solution found, increases limit by 1 and repeats.

- **Complete:** Yes
- **Optimal:** Yes (with uniform costs)
- **Space:** O(b*d) — same as DFS, excellent memory efficiency
- **Time:** O(b^d) — repeated expansions are acceptable overhead

Key Point / Exam Tip

IDS is generally the preferred uninformed search when the depth of the solution is unknown. It combines the best properties of BFS and DFS.

Lecture 16 – Informed Search & Adversarial Search

8.6 Greedy Best-First Search

Definition

Greedy Best-First Search: Expands the node that appears to be closest to the goal based on a heuristic function h(n). h(n) estimates the cost from node n to the nearest goal.

- **Evaluation Function f(n) = h(n)**
- **Complete:** No — can get stuck in loops
- **Optimal:** No — greedily chooses nearest-looking node, may miss optimal path

Example

Romania map example: from Arad to Bucharest, Greedy selects Sibiu (h=253km) over Zerind (h=374km), following heuristic estimates.

8.7 A* Search

Definition

A* Search: Combines actual cost g(n) (cost to reach n from start) with heuristic estimate h(n) (cost to goal from n). Evaluation function: f(n) = g(n) + h(n).

- **Complete:** Yes (with admissible heuristic and finite branches)
- **Optimal:** Yes — if heuristic h(n) is admissible (never overestimates true cost)
- **Admissible Heuristic:** h(n) ≤ h*(n) for all n, where h*(n) is the true cost.

Example

8-puzzle: h(n) = number of misplaced tiles (admissible heuristic). A* uses this to efficiently find the optimal solution.

Route planning: h(n) = straight-line distance to goal. Actual roads may be longer, so it never overestimates → admissible.

Key Point / Exam Tip

A is the most widely used informed search algorithm. It is complete and optimal as long as the heuristic is admissible.*

8.8 Adversarial Search & Minimax Algorithm

Definition

Adversarial Search: Search in the context of competitive multi-agent environments (games) where one agent's gain is another's loss (zero-sum games).

Definition

Minimax Algorithm: A recursive algorithm for choosing the optimal move for a player, assuming the opponent also plays optimally. MAX player tries to maximise utility; MIN player tries to minimise it.

Key Concepts:

- **Terminal State:** End of game — utility is computed here.
- **Utility Function:** Assigns a numeric value to terminal states (e.g., +1 win, 0 draw, -1 loss).
- **Minimax Value:** MINIMAX(s) = UTILITY(s) if s is terminal; MAX over successors if MAX's turn; MIN over successors if MIN's turn.

Example

Tic-Tac-Toe: MAX player is X, MIN player is O. Minimax explores all future game states and picks the move with maximum guaranteed utility.

Chess: Same principle, but with millions of states — requires depth limiting and evaluation functions in practice.

- **Time Complexity:** O(b^m) where b is branching factor and m is max depth.
- **Alpha-Beta Pruning:** Optimisation that prunes branches guaranteed not to affect the final decision, reducing time to O(b^(m/2)) in the best case.

Practice Questions

48. **Q1:** Compare BFS and DFS in terms of completeness, optimality, time, and space complexity.
49. **Q2:** What is Iterative Deepening Search? Why is it considered superior to both BFS and DFS in practice?
50. **Q3:** Explain A* search. What is an admissible heuristic? Give an example.
51. **Q4:** What is the Minimax algorithm? Trace through a simple game tree with branching factor 2 and depth 2, showing how MAX and MIN values propagate.
52. **Q5:** What is Alpha-Beta pruning? How does it improve the efficiency of Minimax without affecting the final result?
53. **Q6:** A robot must navigate a grid from (0,0) to (5,5) with obstacles. Which search algorithm would you recommend and why?

Quick Reference: Search Algorithms Comparison

Algorithm	Complete	Optimal	Time	Space	Type
BFS	**Yes**	Yes*	O(b^d)	O(b^d)	**Uninformed**
UCS	**Yes**	Yes	O(b^C*)	O(b^C*)	**Uninformed**
DFS	**No**	No	O(b^m)	O(bm)	**Uninformed**
IDS	**Yes**	Yes*	O(b^d)	O(bd)	**Uninformed**
Greedy BFS	**No**	No	O(b^m)	O(b^m)	**Informed**
A*	**Yes**	Yes	O(b^d)	O(b^d)	**Informed**
Minimax	**Yes**	Yes	O(b^m)	O(bm)	**Adversarial**

** BFS and IDS are optimal only when all step costs are equal.*